

Landing Pages — Developer Handoff

Six award-grade animated landing pages, handed off for integration into our Shopify build. This guide tells you **what they are, how to run them, and — most importantly — how to actually get them onto Shopify**, because they are *not* Shopify themes out of the box.

Read "The one thing you must understand" before you quote an estimate.

Stack (all six): **Vite + React 18 + Three.js (WebGL) + GSAP + Lenis**. Plain JavaScript (JSX), **no TypeScript, no Tailwind**. CSS is hand-written per project.

0. The one thing you must understand

These pages are **standalone single-page apps** — Vite + React 18 + Three.js + GSAP. They are **not** Shopify Liquid themes, and their "add to cart / price / checkout" is a **local mock** (an in-memory cart with placeholder data). They were built as design + motion references and as fully working front-ends.

So "put it on Shopify" is **not** a copy-paste. There are three real ways to do it (Section 4), from ~a day (iframe embed) to a full theme port. Whichever you pick, **the commerce layer must be rebuilt against Shopify** (Section 5). Everything *visual and motion* is reusable as-is.

If you only take one path: **most stores want Path B — port the animation into the Shopify theme**.

1. The six pages

#	Name	Folder	What it is	Preview port
1	Pink Pill	projects/01-pink-pill/	Animated supplement/pharma landing; persistent 3D capsule + scroll choreography	:8601
2	ALTO	projects/02-alto-penthouse/	Penthouse furniture homepage; WebGL atmosphere, editorial	:8602
3	Atelier Peptide	projects/03-atelier-peptide/	Multi-product research-peptides (v1); traveling particle "compound"	:8603
4	STRATA	projects/04-strata/	Research-peptides (v2); GPU point-cloud that morphs shape; chromatograms	:8604

#	Name	Folder	What it is	Preview port
5	NOCTA	projects/05-nocta/	Research-peptides (v3); glowing self-sequencing DNA ribbon, deep-sea dark	:8605
6	APEXION	projects/06-apexion/	Research-peptides (v10); GPGPU million-point field that boils & crystallizes into instrument readouts	:8607

Live previews run on a private Tailscale network behind a passcode. To view them you need (a) to be added to the Tailscale net and (b) the passcode — **ask the project owner for both**. You do **not** need the previews to build; everything is in the source zips.

Preview host pattern: `https://andreidutescus-macbook-pro.tailb4cf4e.ts.net:<port>`

2. Prerequisites

- **Node.js 20 LTS** (18+ works). Safe target is 20 LTS.
- **npm** (bundled with Node). Lockfiles (`package-lock.json`) are included — use `npm ci` for reproducible installs.
- A browser with **WebGL2** (all evergreen browsers; Safari 15+). WebGL powers the hero animation, but every page has a **non-WebGL fallback** (Section 6).

No paid dependencies, no private registries. GSAP is the **free/standard** build only — **no Club GreenSock plugins** — so no special GSAP license is needed beyond the standard one.

3. Run any page locally (identical for all six)

```
cd projects/06-apexion      # or any project folder
npm ci                      # exact locked install (first time)
npm run dev                 # hot-reload dev server -> prints a localhost URL
```

Produce a **deployable static build**:

```
npm run build               # static site into ./dist
npm run preview             # serve ./dist locally to sanity-check
```

`dist/` is a plain static bundle (HTML + hashed JS/CSS/font assets). Host it on **any** static host (Netlify, Vercel, Cloudflare Pages, S3+CloudFront). This matters for Path A.

Cache gotcha: every build emits new hashed filenames. When re-deploying to a static host, serve `index.html` with `Cache-Control: no-cache` (hashed assets can stay immutable). Otherwise a browser holding an old `index.html` 404s on deleted asset hashes and renders **blank**. Don't let this bite you.

4. Getting it onto Shopify — three paths

Path A — Embed the built page (fastest, ~0.5–1 day)

`npm run build`, host `dist/` on a static host, embed it in a Shopify page via an `<iframe>` (custom section/template or app-embed block).

- **Pros:** keeps 100% of the animation with near-zero rework; decoupled from the theme.
- **Cons:** it's an iframe — no shared scroll with the store, trickier mobile, weaker SEO for that content, and **the cart is separate**: repoint the mock "Add to cart" buttons to real Shopify cart deep-links (`/cart/add?id=<variantId>`) or `postMessage` to the parent theme which calls the Cart AJAX API.
- **Good for:** a hero/campaign/landing page that isn't the primary PDP or collection grid.

Path B — Port the animation into the Shopify theme (recommended, ~1–3 weeks/page)

Keep the **WebGL canvas + GSAP choreography** as theme assets, rebuild the page's DOM as **Liquid sections** fed by real Shopify data, use the **native cart + checkout**.

1. Move the compiled animation bundle (or port the Three.js/GSAP modules) into `assets/`, loaded only on the target template.
 2. Recreate each section (hero, catalog, verify, order, FAQ...) as a **Liquid section** with schema settings so content is theme-editor editable.
 3. Replace mock product data (`src/lib/data.js`) with **Shopify products / collections / metafields**.
 4. Wire "Add to cart" to the **Shopify AJAX Cart API** (`POST /cart/add.js`) and use the real checkout. Delete the mock cart (Section 5).
 5. Keep WebGL as a fixed background canvas behind the Liquid content — the pages are already built this way (one persistent canvas; content floats above it).
- **Pros:** real Shopify cart/checkout/SEO/theme-editor; the animation survives intact.
 - **Cons:** the most integration work; someone owns the Three.js/GSAP layer.
 - **Good for:** the primary storefront / a page that must convert and rank.

Path C — Headless (Shopify Hydrogen / React, highest effort)

Because these are already React, the **components port most directly** into a **Hydrogen** (React) storefront. Animation code moves nearly verbatim; swap the mock cart for Hydrogen's cart and the **Storefront API** for product data; deploy on Oxygen (or any Node host).

- **Pros:** React components + animation transfer with the least rewrite; best long-term DX.
- **Cons:** headless is a bigger commitment (hosting, routing, SEO, Storefront-API checkout).
- **Good for:** a from-scratch headless rebuild.

Recommendation: Path A to ship a campaign page fast; **Path B for the real store**; Path C only if the whole store is going headless.

5. What you MUST replace (mock commerce → Shopify)

Every page ships a **fake, in-memory commerce layer** — realistic but 100% local. Before any page goes live, swap these for Shopify:

Mock (in the source)	Replace with (Shopify)
<code>src/lib/data.js</code> — names, prices, doses, purities	Shopify products / variants / metafields (or Storefront API in headless)
<code>src/lib/cart.jsx</code> — <code>CartProvider</code> / <code>useCart</code> (in-memory)	Shopify Cart AJAX API (<code>/cart/add.js</code> , <code>/cart.js</code>) or Hydrogen cart
<code>components/StickyCart.jsx</code> , "Add — \$X" buttons	Real add-to-cart + the store's cart drawer/checkout
Hard-coded prices in the UI	Shopify price fields (respect currency/locale)
"Checkout" (no-op)	Shopify checkout

Prices and product facts in these demos are placeholders — do not ship them as real offers. Point buy buttons at real variants and let Shopify own price, tax, and checkout.

Compliance (peptide pages 3–6): these carry **"For laboratory research use only — not for human or veterinary use"** and related RUO disclaimers at buy points and in the footer. If those products go live, **keep that language** and confirm it with whoever owns compliance — it is load-bearing, not decoration.

6. Robustness laws — do NOT strip these when porting

Each page is hardened against real failure modes. Preserve these behaviors when refactoring into Shopify (they're cheap and prevent "blank page" tickets):

- `prefers-reduced-motion` → renders a static, no-autoplay version. Test with the OS "Reduce motion" setting on.
- **Never-blank WebGL** → if WebGL fails or the GPU sim can't run, a **static SVG/CSS fallback** renders instead of an empty canvas. Keep the fallback path.

- **Mobile** → lighter/stacked layout, reduced particle counts, no custom cursor on touch.
 - **Cleanup** → animations, tickers, observers, and WebGL contexts dispose on unmount (prevents leaks under SPA navigation).
 - **QA hooks (dev only)** → append `?rm=1` (force reduced motion) or `?nogl=1` (force the non-WebGL fallback) to any page URL to exercise those paths.
-

7. Per-project quick reference

Each project folder has its own **HANDOFF.md** with its stack, entry points, the single file that owns its signature animation, and its preview port. Start there for that page.

Shared architecture (all six): `index.html` → `src/main.jsx` (mounts React) → `src/App.jsx` (composes sections) → `src/components/*` (DOM sections) + `src/three/*` (WebGL scene) + `src/lib/*` (scroll, cart, data, helpers). One persistent `<canvas>` is fixed behind the content; a global GSAP ScrollTrigger drives the whole page.

8. Licensing / assets

- **Fonts** bundled via `@fontsource*` (open licenses — OFL/Apache); ship with the build, nothing external to load.
 - **three.js, @react-three/fiber & drei, Lenis, split-type** — all MIT.
 - **GSAP** — standard/free build, **no Club plugins**. Standard GSAP license applies (covers standard commercial web use).
 - The animations need **no third-party image/video assets** (generated in-shader / in-SVG). Product photography, if any, is yours to supply.
-

9. Questions

For Tailscale access + the preview passcode, and for intended design/behavior questions, contact the project owner. **Live previews are the source of truth for how it should look and move; the zips are the source of truth for the code.**